数据库事务（Transaction）详解

一、事务的基本概念

1.1 事务的定义

事务是 **数据库操作的一个逻辑单元** ，它包含一系列对数据库的访问和更新操作，这些操作要么全部执行，要么全部不执行。

举例说明 ：

```
-- 银行转账事务示例
BEGIN TRANSACTION
UPDATE account SET money = money - 50 WHERE no = "A"
UPDATE account SET money = money + 50 WHERE no = "B"
COMMIT
```

1.2 事务的ACID特性（重点）

真题回顾 ：文档1简答题第2题（5分）简述事务的特性。

（1）原子性（Atomicity）

- 事务中的所有操作要么全部完成，要么全部不完成
- 中间状态对系统不可见
- **举例** ：转账事务中，如果第二步失败，第一步的扣款必须撤销

（2）一致性（Consistency）

- 事务执行必须使数据库从一个一致性状态变到另一个一致性状态
- 保持数据库的完整性约束
- **举例** ：转账前后，A和B账户总额保持不变

（3）隔离性（Isolation）

- 并发执行的事务相互隔离，互不干扰
- 一个事务的中间结果对其他事务不可见
- **举例** ：T1转账过程中，T2查询总额时不应看到中间的不一致状态

（4）持久性（Durability）

- 事务提交后，对数据库的修改是永久的
- 即使系统故障也不会丢失

二、事务的状态转换

课件图示 ：文档2第17.13页的状态图

事务可能的状态：

- **活跃（Active）** ：正在执行
- **部分提交（Partially Committed）** ：最后语句已执行
- **提交（Committed）** ：成功完成
- **失败（Failed）** ：无法正常继续
- **中止（Aborted）** ：已回滚

三、事务调度与并发控制

3.1 调度类型

（1）串行调度

- 一个事务完全执行完后才开始下一个事务
- **优点** ：保证一致性
- **缺点** ：并发度低

例题1 ：判断以下是否为串行调度

```
T1: read(A), write(A), read(B), write(B)
T2: read(A), write(A), read(B), write(B)
调度：先执行完T1所有操作，再执行T2所有操作
```

答案 ：是串行调度

（2）并发调度

- 多个事务的操作交叉执行
- **优点** ：提高资源利用率
- **挑战** ：可能破坏一致性

3.2 并发调度的问题（三类数据不一致性）

（1）丢失修改（Lost Update）

问题描述 ：两个事务同时修改同一数据，后提交的修改覆盖了先提交的修改。

例题2 ：分析以下调度的错误

```
T1: read(A), A=A-50, write(A)
T2: read(A), A=A+100, write(A)
调度：
T1读A=100 → T2读A=100 → T1写A=50 → T2写A=200
```

结果 ：T1的修改丢失，最终A=200而不是150

（2）不可重复读（Non-repeatable Read）

问题描述 ：同一事务内两次读取同一数据，结果不同。

subtypes ：

- 值改变：第二次读取时值已被修改
- 幻影读：第二次读取时出现新记录或记录消失

例题3 ：幻影现象示例

```
-- T1第一次查询
SELECT * FROM students WHERE age = 20; -- 返回3条记录

-- T2执行插入
INSERT INTO students VALUES(4, '张三', 20);

-- T1第二次相同查询
SELECT * FROM students WHERE age = 20; -- 返回4条记录
```

（3）读"脏"数据（Dirty Read）

问题描述 ：读取了其他事务未提交的修改，而该事务后来回滚。

例题4 ：脏读示例

```
T1: read(A), A=A+100, write(A), rollback
T2: read(A), print(A)
调度：T1写A后，T2读取A=200，然后T1回滚，A恢复为100
```

结果 ：T2读到了不存在的数据

四、可串行化（Serializability）

4.1 冲突可串行化

冲突操作定义（课件第17.37页）：

- read(Q)和write(Q)冲突
- write(Q)和read(Q)冲突
- write(Q)和write(Q)冲突
- read(Q)和read(Q)不冲突

冲突等价 ：通过交换非冲突操作可以转换成串行调度

4.2 前驱图测试法

步骤 ：

1. 为每个事务创建节点
2. 如果Ti的操作在Tj的冲突操作之前，添加边Ti→Tj
3. 如果图无环，则调度是冲突可串行化的

例题5 ：判断以下调度是否冲突可串行化

```
T1: read(A), write(A)
T2: read(A), write(A), read(B), write(B)
T3: read(B), write(B)

调度：
T1读A → T2读A → T1写A → T3读B → T2写A → T2读B → T3写B → T2写B
```

解答 ：

- T1→T2（T1写A在T2读A之后，但T1读A在T2读A之前？需要仔细分析冲突）
- 实际应该构造前驱图来判断

五、恢复机制

5.1 事务日志

真题回顾 ：文档1选择题第9题（2分）事务日志的用途是数据恢复

日志记录类型：

- 开始事务
- 更新操作（前像和后像）
- 提交事务
- 中止事务

5.2 系统故障恢复策略

真题回顾 ：文档1简答题第4题（5分）简述系统故障时的数据库恢复策略

恢复步骤 ：

1. **正向扫描** ：确定REDO和UNDO队列
2. **UNDO处理** ：回滚未完成事务
3. **REDO处理** ：重做已提交事务

六、隔离级别

6.1 四级隔离级别

真题相关 ：文档2第17.51页详细说明

隔离级别	脏读	不可重复读	幻影读
读未提交	可能	可能	可能
读已提交	不可能	可能	可能
可重复读	不可能	不可能	可能
可串行化	不可能	不可能	不可能

6.2 SQL中的事务控制

语法示例 ：

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
BEGIN TRANSACTION;
-- 事务操作
COMMIT;
-- 或 ROLLBACK;
```

七、重点例题精讲

例题6：综合应用题（参考文档1综合题）

问题 ：关系模式R(商店编号S,商品编号I,数量N,部门编号D,负责人M)，函数依赖：

- (S,I) → D
- (S,D) → M
- (S,I) → N

求解 ：

1. 候选码：(S,I)
2. 判断范式：存在部分依赖？不存在，因为(S,I)是单属性码
3. 判断BCNF：检查每个函数依赖左边是否都是超码

例题7：并发调度分析

分析以下调度是否存在问题：

```
T1: read(A), A=A-100, write(A)
T2: read(A), read(B), print(A+B)
T3: read(B), B=B+50, write(B)
```

调度序列：
T1读A → T2读A → T3读B → T1写A → T2读B → T2打印 → T3写B

分析要点 ：

- T2读取A时是旧值，读取B时可能已被T3修改？
- 是否存在不可重复读或脏读？

八、真题重点梳理

根据文档1的考试内容，重点掌握：

1. **选择题** ：事务特性、锁机制、恢复技术
2. **填空题** ：ACID特性、隔离级别、恢复策略

3. **简答题** ：事务特性、恢复流程、并发问题
4. **综合题** ：调度分析、范式判断、E-R图设计

九、学习建议

1. **理解核心概念** ：重点掌握ACID特性和三类并发问题
2. **多做练习** ：特别是调度分析和冲突判断
3. **结合实际** ：理解每个概念在真实数据库中的应用
4. **注意细节** ：区分不同隔离级别的具体表现

需要我进一步详细讲解某个特定知识点或提供更多例题吗？